SOMAIYA
VIDYAVIHAR

K J Somaiya Institute of Technology
An Autonomous Institute Permanently Affiliated to the University of Mumbai

## DEPARTMENT OF INFORMATION TECHNOLOGY

**Course Name and Code:** Data Structures Lab **(**ITL302)
**Semester:** III (SYIT)
**Academic Year:** 2026-27 (Odd Semester)
**Name:** Rashi Dawda
**Roll No:** 23
**Batch:** AS2

### Experiment No. 01

**Aim:** Implementation of Stack using an array for real-world application.

**Objectives:**

1. To introduce the concepts of data structures and analysis procedure.
2. To conceptualize linear data structures and its implementation for various real-world applications.

**Theory:**

**1. Introduction to Stack**

A **Stack** is a linear data structure that works on the **LIFO (Last In, First Out)** principle — the last element added is the first one to be removed. A simple way to picture this is a stack of plates: you place a new plate on top, and whenever you need one, you pick it up from the top too. All insertions and deletions happen from just one end, called the **top** of the stack. It can be implemented using an **array** (fixed size, with a **top** variable keeping track of the last inserted element) or a **linked list** (which can grow and shrink dynamically). In this experiment, the array-based method is used, where `MAX` defines the maximum number of elements the stack can hold, and the top starts at `-1` to indicate that the stack is initially empty.

**2. Various Operations**

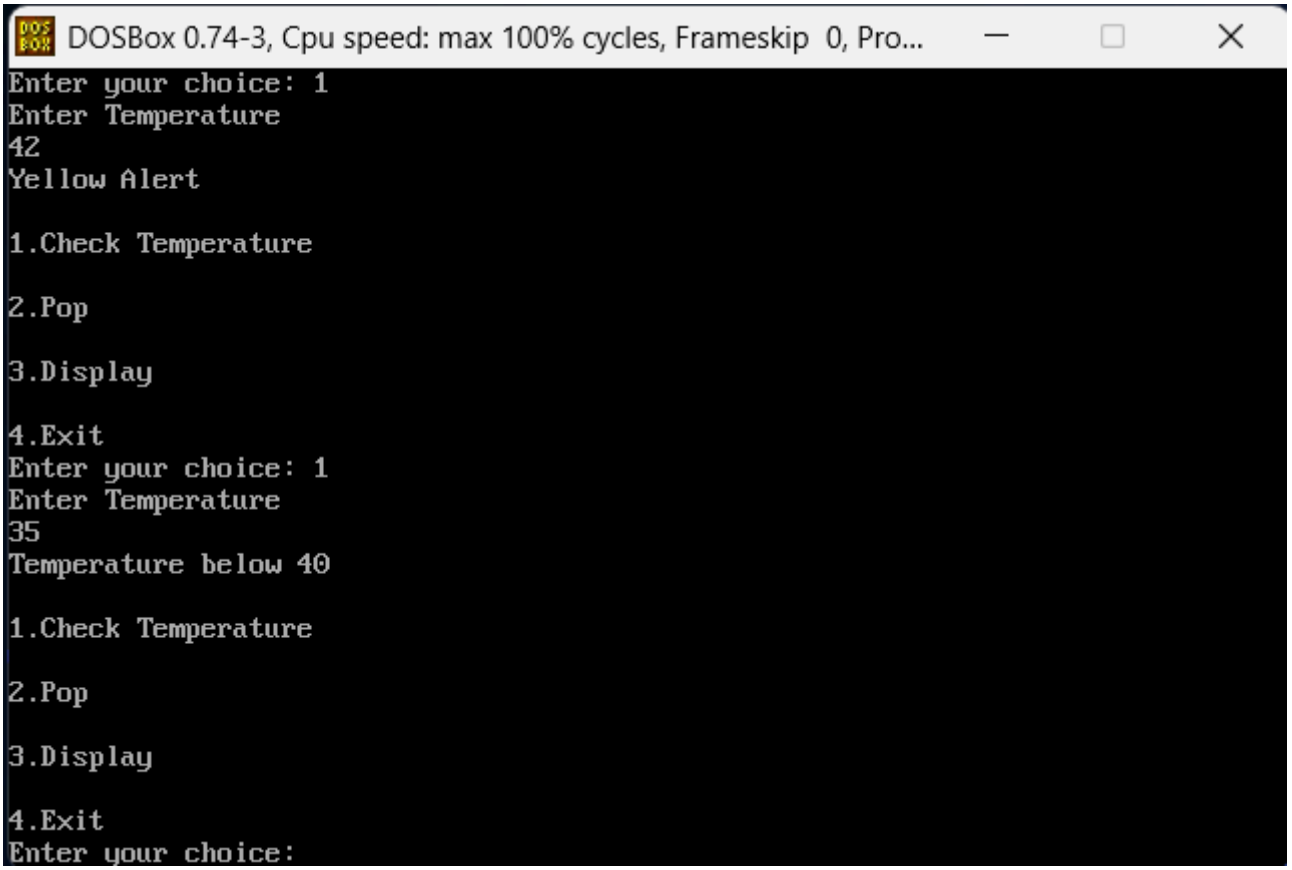
- **PUSH** – Adds a new element to the top of the stack. If the stack is already full (`top == MAX-1`), it results in a **stack overflow**.
- **POP** – Removes the topmost element from the stack. If the stack is empty (`top == -1`), it results in a **stack underflow**.
- **PEEK** – Lets you view the value of the top element without actually removing it from the stack.
- **DISPLAY** – Prints all the elements currently present in the stack, usually from the top down to the bottom.

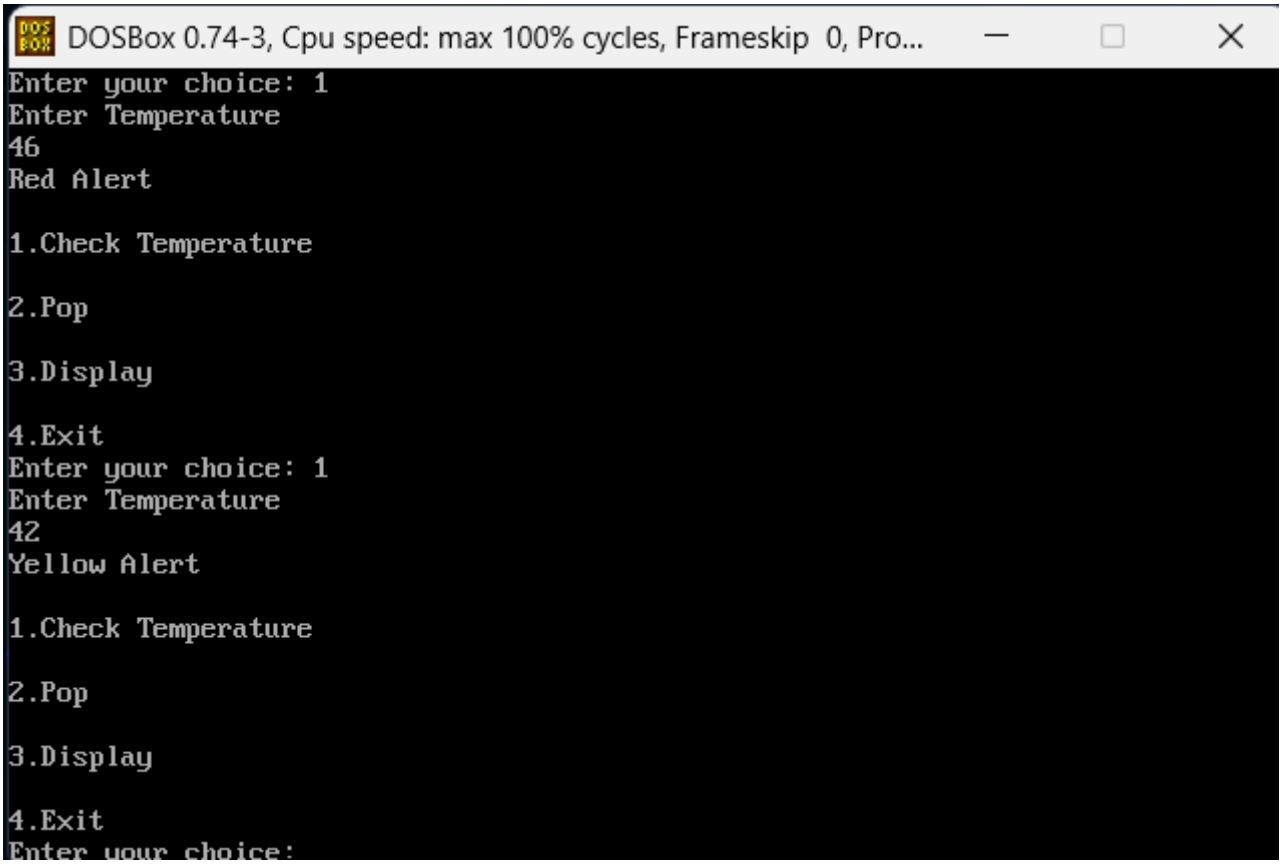
**Conclusion:**

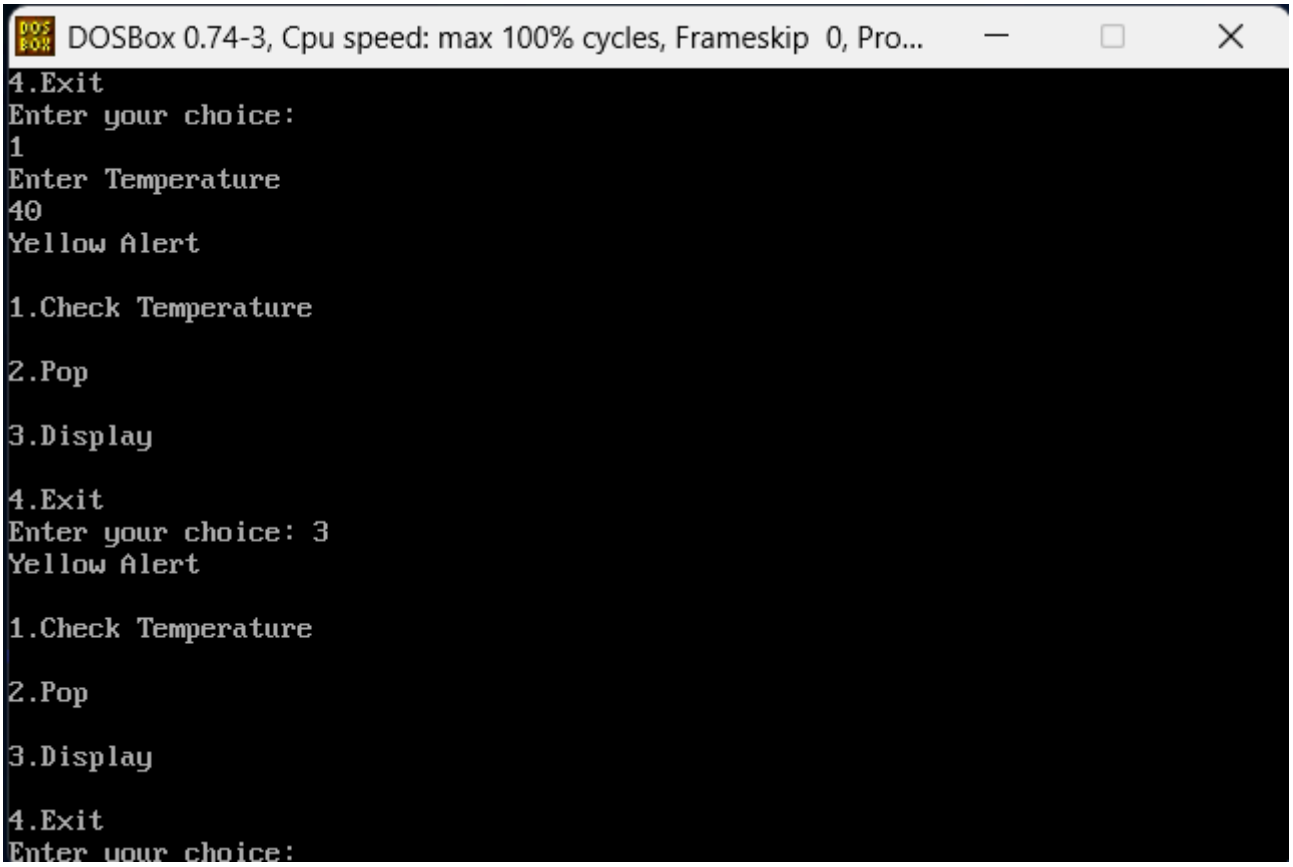
Doing this experiment helped me actually understand how a stack works instead of just reading about LIFO in theory. Writing the PUSH and POP functions made it clear why we need to check for overflow and underflow before adding or removing elements. The temperature monitoring example was a nice touch too, since it showed how something as simple as a stack can be used to track the latest reading and flag alerts in a real system.

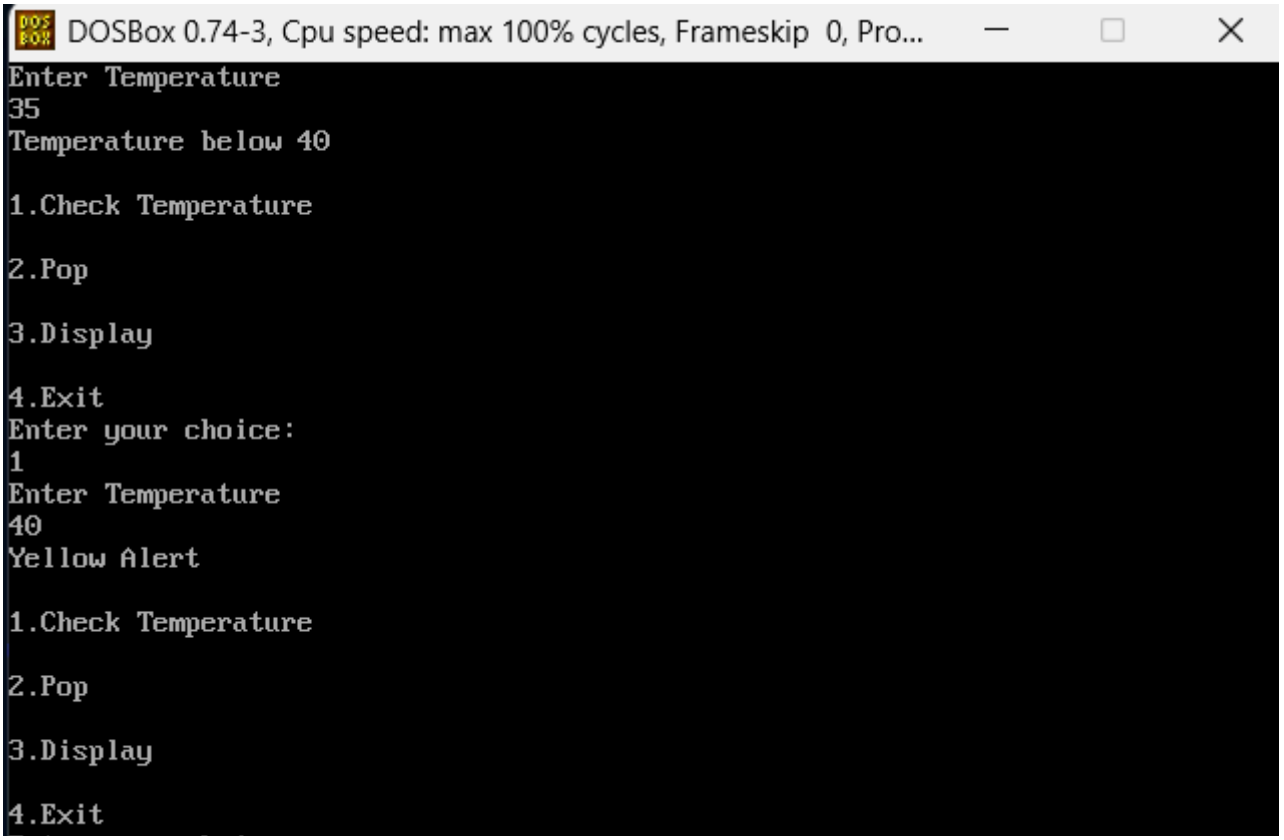
**Outcome:**

Apply the concepts of stacks for real-world applications.

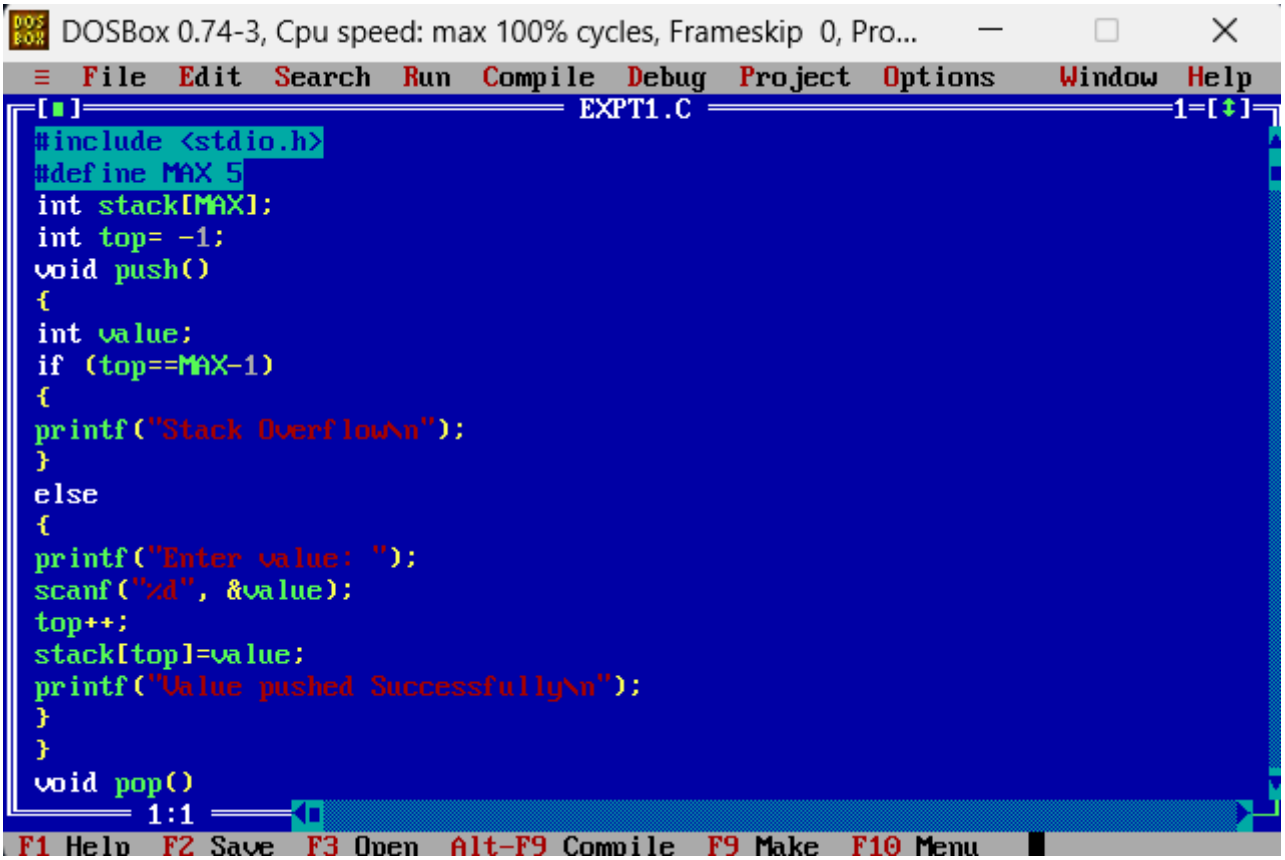

DOSBox 0.74-3, Cpu speed: max 100% cycles, Frameskip 0, Pro...
Enter your choice: 1
Enter Temperature
42
Yellow Alert
1.Check Temperature
2.Pop
3.Display
4.Exit
Enter your choice: 1
Enter Temperature
35
Temperature below 40
1.Check Temperature
2.Pop
3.Display
4.Exit
Enter your choice:

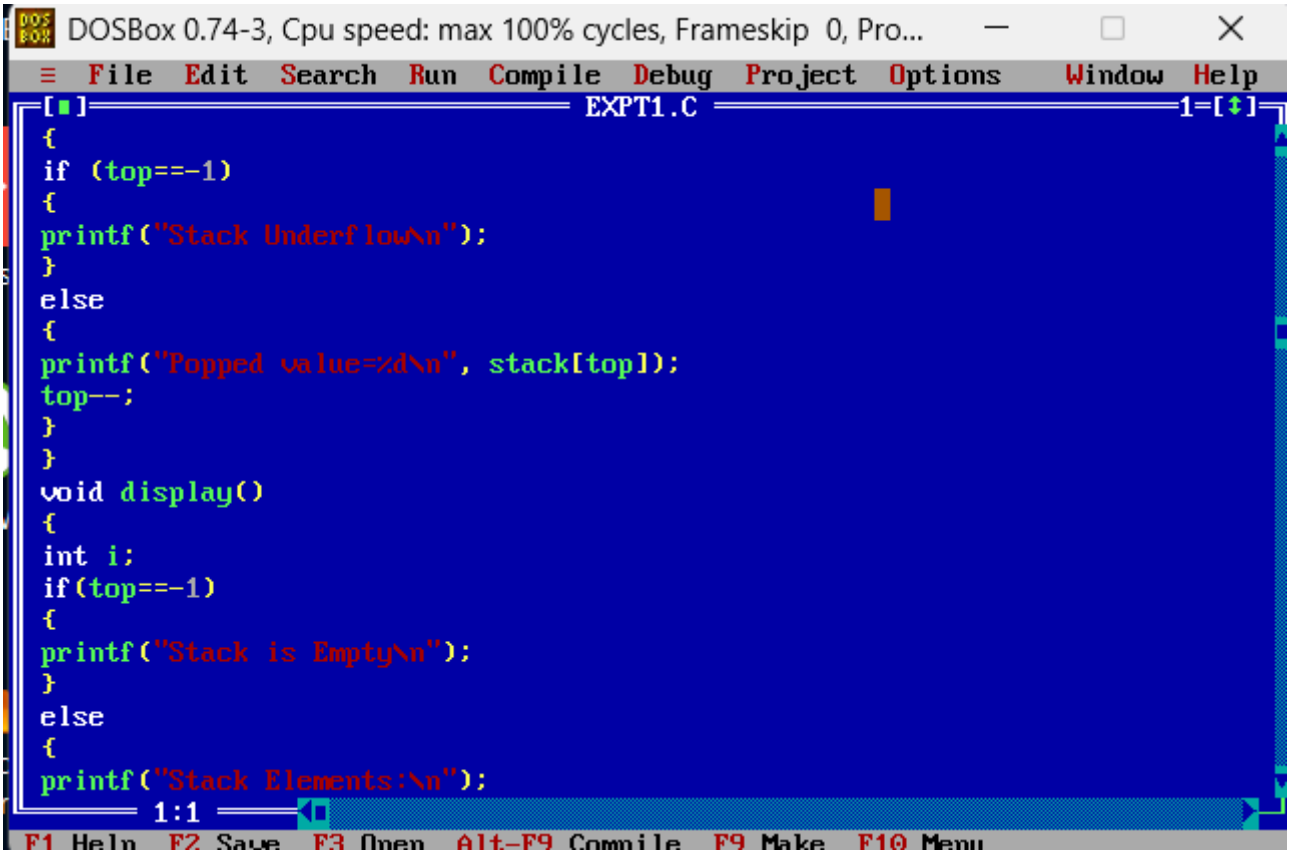

DOSBox 0.74-3, Cpu speed: max 100% cycles, Frameskip 0, Pro...
Enter your choice: 1
Enter Temperature
46
Red Alert
1.Check Temperature
2.Pop
3.Display
4.Exit
Enter your choice: 1
Enter Temperature
42
Yellow Alert
1.Check Temperature
2.Pop
3.Display
4.Exit
Enter your choice:

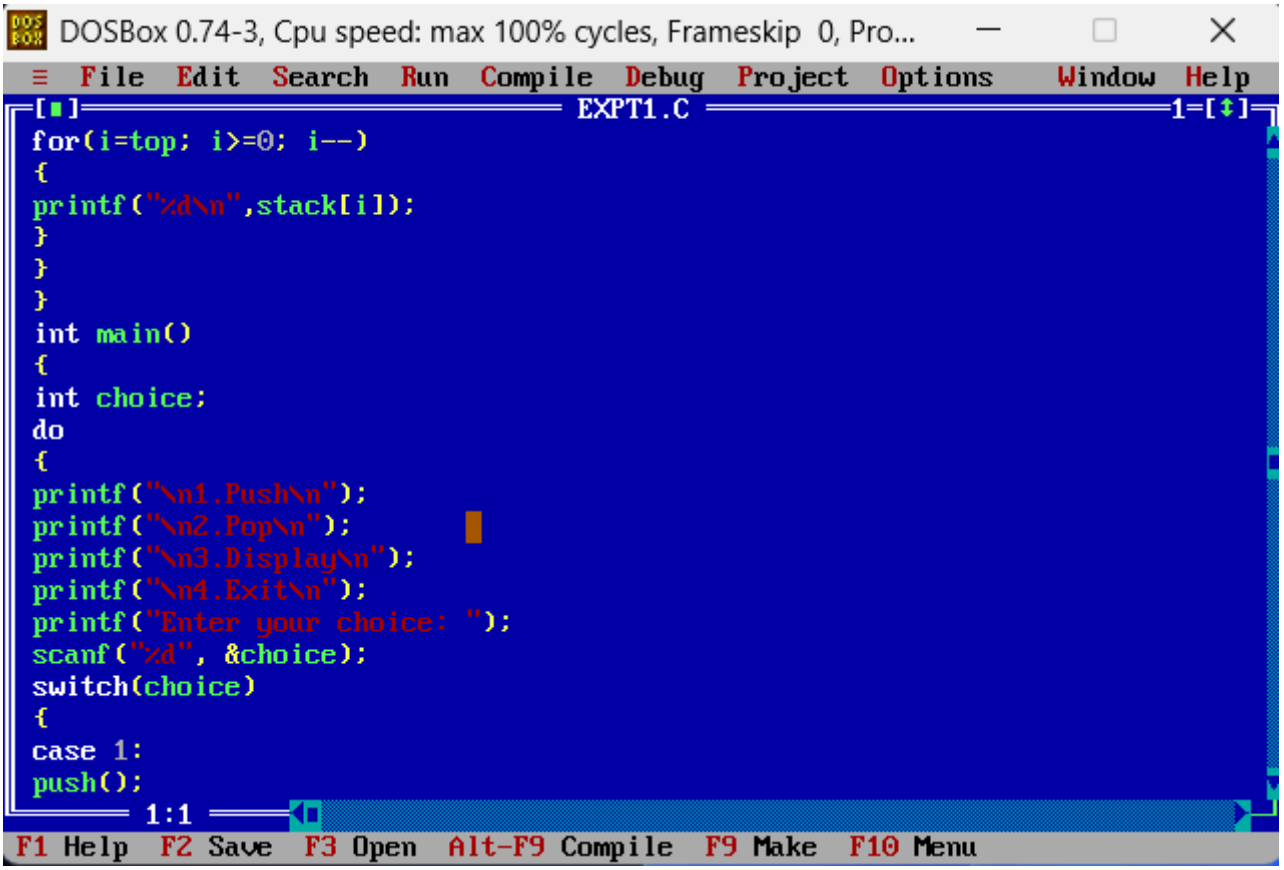

DOSBox 0.74-3, Cpu speed: max 100% cycles, Frameskip 0, Pro...
4.Exit
Enter your choice:
1
Enter Temperature
40
Yellow Alert
1.Check Temperature
2.Pop
3.Display
4.Exit
Enter your choice: 3
Yellow Alert
1.Check Temperature
2.Pop
3.Display
4.Exit
Enter your choice:

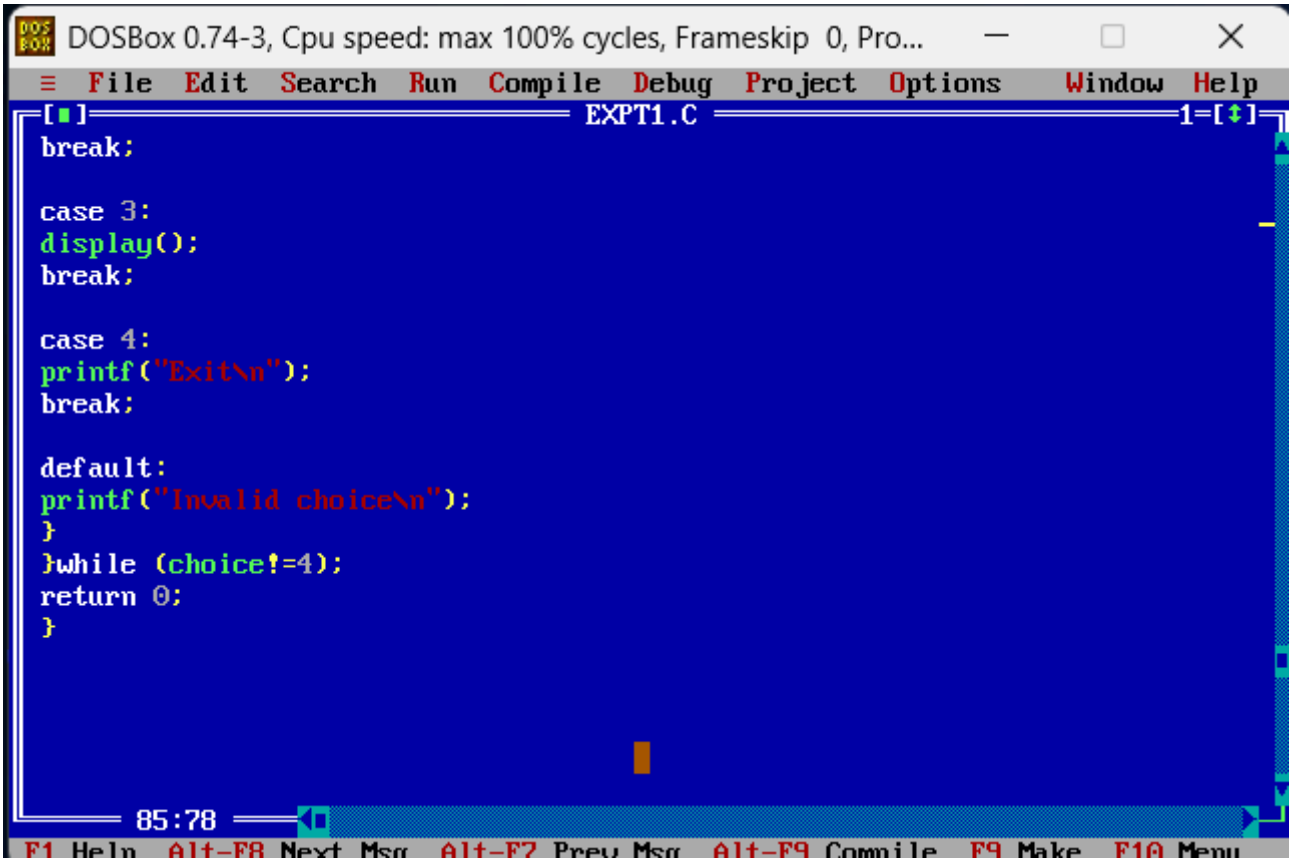

DOSBox 0.74-3, Cpu speed: max 100% cycles, Frameskip 0, Pro...
Enter Temperature
35
Temperature below 40
1.Check Temperature
2.Pop
3.Display
4.Exit
Enter your choice:
1
Enter Temperature
40
Yellow Alert
1.Check Temperature
2.Pop
3.Display
4.Exit

**CODE:**


DOSBox 0.74-3, Cpu speed: max 100% cycles, Frameskip 0, Pro...
File Edit Search Run Compile Debug Project Options Window Help
EXPT1.C
#include <stdio.h>
#define MAX 5
int stack[MAX];
int top= -1;
void push()
{
int value;
if (top==MAX-1)
{
printf("Stack Overflow\n");
}
else
{
printf("Enter value: ");
scanf("%d", &value);
top++;
stack[top]=value;
printf("Value pushed Successfully\n");
}
}
void pop()
1:1
F1 Help F2 Save F3 Open Alt-F9 Compile F9 Make F10 Menu


DOSBox 0.74-3, Cpu speed: max 100% cycles, Frameskip 0, Pro...
File Edit Search Run Compile Debug Project Options Window Help
EXPT1.C
{
if (top==-1)
{
printf("Stack Underflow\n");
}
else
{
printf("Popped value=%d\n", stack[top]);
top--;
}
}
void display()
{
int i;
if(top==-1)
{
printf("Stack is Empty\n");
}
else
{
printf("Stack Elements:\n");
1:1
F1 Help F2 Save F3 Open Alt-F9 Compile F9 Make F10 Menu


DOSBox 0.74-3, Cpu speed: max 100% cycles, Frameskip 0, Pro...
File Edit Search Run Compile Debug Project Options Window Help
EXPT1.C
for(i=top; i>=0; i--)
{
printf("%d\n",stack[i]);
}
}
}
int main()
{
int choice;
do
{
printf("\n1.Push\n");
printf("\n2.Pop\n");
printf("\n3.Display\n");
printf("\n4.Exit\n");
printf("Enter your choice: ");
scanf("%d", &choice);
switch(choice)
{
case 1:
push();
1:1
F1 Help F2 Save F3 Open Alt-F9 Compile F9 Make F10 Menu


DOSBox 0.74-3, Cpu speed: max 100% cycles, Frameskip 0, Pro...
File Edit Search Run Compile Debug Project Options Window Help
EXPT1.C
break;
case 3:
display();
break;
case 4:
printf("Exit\n");
break;
default:
printf("Invalid choice\n");
}
}while (choice!=4);
return 0;
}
85:78
F1 Help Alt-F8 Next Msg Alt-F7 Prev Msg Alt-F9 Compile F9 Make F10 Menu